

Agent Excellence

Kickstarter workshop

This handout is your take-home reference for designing, building, and operating excellent AI agents. Use it during the workshop to capture your decisions, and after the workshop as a reusable blueprint for every agent you build.

Your name:

Date:

Organization:

Agent Excellence

Kickstarter workshop

Agent Excellence is a practical, architecture-first workshop that helps teams design AI agents that are useful, safe, and scalable.

Rather than starting with tools or features, we focus on the design decisions that determine whether an agent succeeds in real environments.

You will learn how to:

- Decompose a real business use case into an agent design
- Make explicit decisions about scope, autonomy, and risk
- Translate decisions into clear, testable agent instructions
- Evaluate and refine your agent design before building at scale

THE LEARNING CYCLE

LEARN

APPLY

TEST

REFINE

Each module introduces a concept, applies it to your use case, tests the outcome, and refines it. You will loop through this cycle multiple times throughout the day.

What you will leave with

- A first-pass instruction set (purpose, non-goals, tone, escalation rules)
- A boundary model (answer / ask / act / stop)
- A knowledge plan (what the agent may trust and when to say "I don't know")
- A clear view of tooling and integrations required
- A starter evaluation set you can run on every change
- A lightweight operating plan for post-launch improvement

Together, these form your Agent Excellence Blueprint.

Kickstarter workshops

Workshops generally follow either a **presentation + lab format**, or a **principles-based approach**. Kickstarter workshops use the principles-based method, focusing on understanding the 'why' through discussions – ideal for fostering strategic, adaptable insights.

Presentation + lab content

Balanced learning approach

A mix of presentation-heavy segments and hands-on labs offer conceptual understanding and practical application.

Structured, step-by-step guides

Participants receive clear instructions for each lab task allowing them to follow along at their own pace with guided support.

Reinforces key concepts

The flow from theory to proactive reinforces learning, helping participants see immediate application of what they just learned.

Principles-focused

Principles-driven learning

Emphasizes understanding the core principles and 'why' behind the processes

Collaborative and conversation heavy

Sessions center around group discussions, encouraging participants to share perspectives and explore topics deeply together.

Minimal lab, maximum reflection

Rather than focusing on hands-on-labs, this workshop prioritizes reflective exercises and activities that encourage participants to think critically about real-world

Why we are here...

The reality of building agents

Most AI agents fail not because of models or AI capabilities, but because of unclear design decisions.

Agent Excellence focuses on the decisions that determine whether an agent is:

- Useful in real workflows
- Predictable and safe
- Scalable beyond a demo

This workshop does not start with tools, features, or prompts. It starts with reality: how work actually happens, where friction exists, and where agents can create meaningful improvement.

You will spend more time thinking, deciding, testing, and refining than configuring tools.

WHAT HAPPENS WHEN INSTRUCTIONS ARE UNCLEAR

- Scope creep — the agent tries to do too much
- Unpredictable behavior — routing and responses vary
- Incorrect information — unsafe over-generalization
- Loss of trust and maintainability

“Unclear instructions don’t just make agents slightly worse — they change their shape. Ambiguity gets treated as permission.”

Workshop flow



Define your use case

- Key scenarios
- Success metrics
- Target users



Choose your platform

- Choose the right development platform based on capabilities required



Write clear instructions

Add instructions that describe primary purpose, explicit non-goals, success definition and tone



Connect to the right tools

Add connectors, agent flows, or APIs that allow the agent to take action and connect to other systems



Configure knowledge sources

Add approved knowledge sources, citation expectations, when to say "I don't know"



Refine boundaries

Add boundaries to define deterministic logic and decisions the agent can't make or must escalate



Evaluate your agent

Test the agent with real scenarios to evaluate accuracy, variations, and edge cases



Publish & Monitor

Deploy the agent to approved channels and collect usage data and feedback

↻ At every step: Apply → Evaluate → Test → Refine

Framing



Review your use case

Purpose of this section

This section helps you **pressure-test your starting idea** before you design or build an agent.

The goal is not to perfect the solution, but to:

- Clarify *what problem you are actually solving*
- Make success measurable
- Narrow the audience
- Make intentional design choices about agent type and platform

Reminder

Ambiguity here almost always leads to fragile agents later.



Define Your Use Case

Describe your agent use case in one sentence. Be specific about who it helps, what it does, and what outcome it delivers.

ONE-SENTENCE USE CASE

The HR onboarding assistant helps new employees and hiring managers find trusted onboarding information, complete required tasks, and understand what to do next in the first 30 days.

Reality-check prompts:

- Is this something people actually do today (manually)?
- Can you name at least 3 specific people who would use this?
- What happens today when this goes wrong?
- Is the data the agent needs actually available?
- Would a simpler solution (form, flow, FAQ page) work instead?

Primary inputs (what does the agent receive?):

INPUTS

Employee role, start date, location, department. Policy and benefits documentation, training requirements, checklists, task requests

Primary outputs (what does the agent produce?):

OUTPUTS

Clear answers to onboarding questions, step-by-step guidance, links, next actions, reminders or summaries

Target users:

DIRECT AND INDIRECT USERS

New employees, hiring managers, HR operations team, IT support, learning and compliance, payroll or benefits teams

Value Statement & Success Metrics

Write a clear value statement. What measurable improvement will this agent create? How will you know it's working?

Use this template to create a crisp value statement. Avoid vague benefits like 'saves time'.

Template: "For [user], when [moment/trigger], the agent will [specific benefit]."

VALUE STATEMENT

For new employees and hiring managers, when onboarding tasks or questions arise during the first 30 days, the agent will provide trusted guidance, next steps, and escalation support so onboarding is completed faster and with fewer missed actions.

Success metrics (be specific and measurable):

e.g., reduce resolution time by 30%, handle 80% of Tier-1 inquiries without escalation...

- Reduce routine onboarding question to HR by 30% within 3 months
- Reduce average time to find onboarding information from 20min to 5min
- Achieve 90% completion within first two weeks

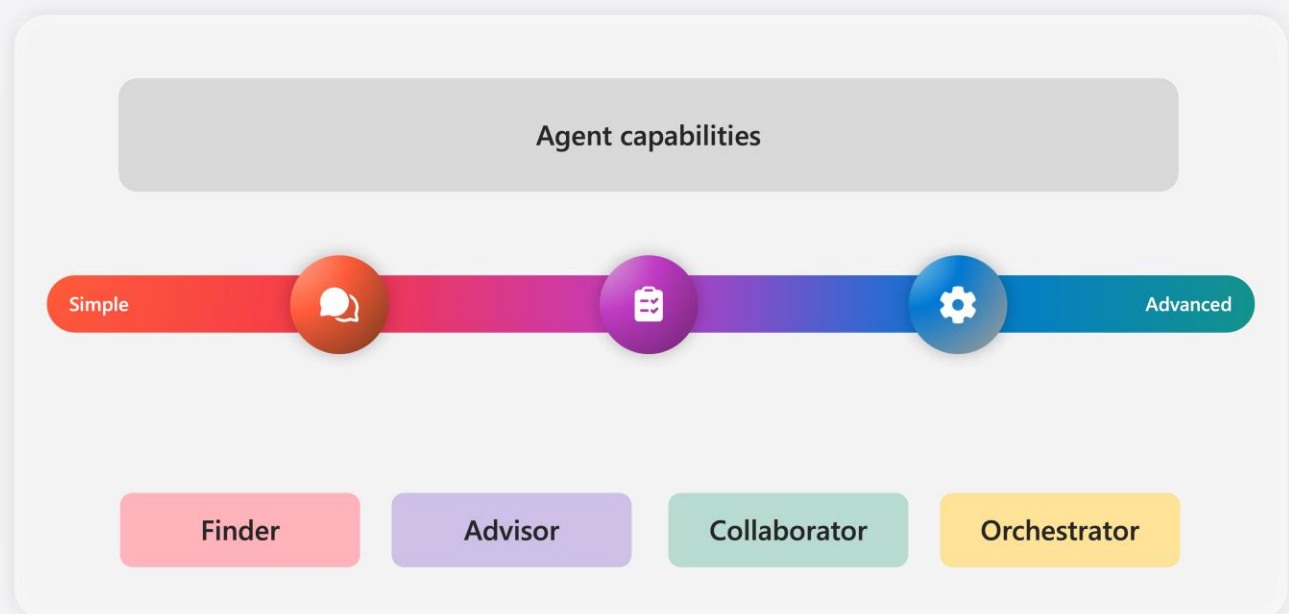
Peer feedback on my use case (after read-back):

What did your partner understand? What was unclear?

Choosing the right agent type

Agent types describe the role an agent plays, not the technology behind it.

Use them to clarify what the agent is responsible for, how much autonomy it has, and what design, risk and governance considerations apply



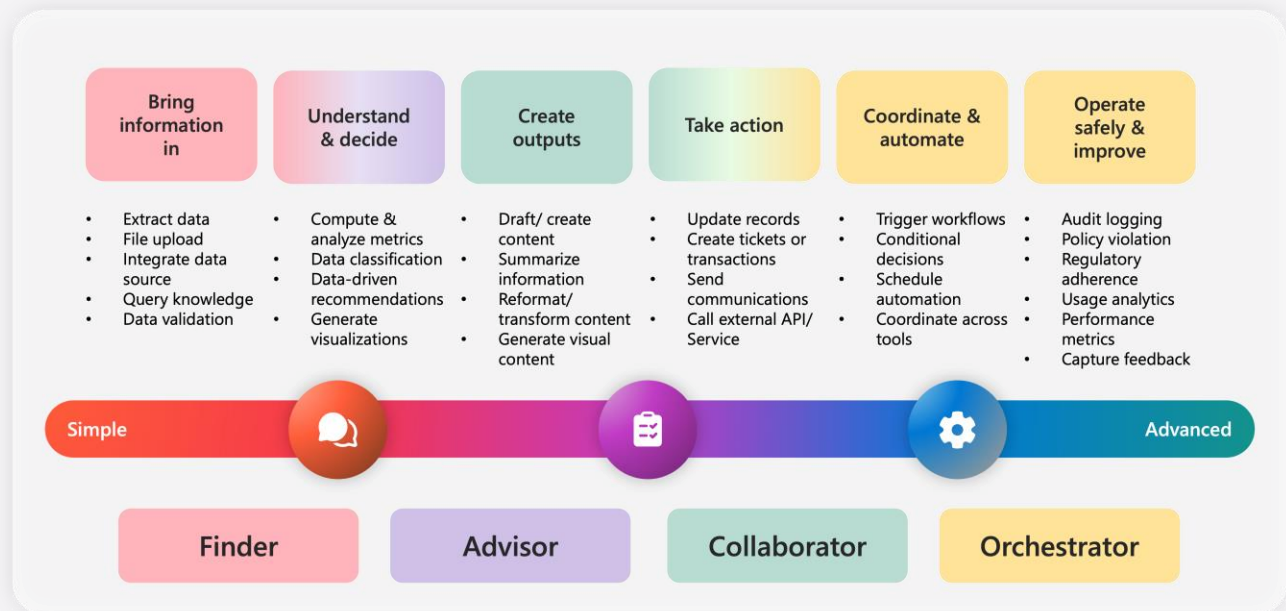
Not every friction point needs the same kind of agent. Some opportunities are best addressed by an agent that helps people find information quickly. Others need an agent that provides guidance, supports collaboration, or coordinates a sequence of actions across systems and teams.

Selecting the right agent type helps teams become clearer about what the agent is actually there to do. It creates a more practical conversation about scope, capability, and fit. It also helps shape later design choices, such as what knowledge the agent needs, how much autonomy is appropriate, and where human oversight should remain.

In this method, agent types are used to match the nature of the problem to the role the agent should play. This helps teams form stronger concepts earlier and avoid designing solutions that are too limited for the need, or too ambitious for the context.

How agent types differ in capability

Agent types do not only differ in name. They also differ in the kind of work they are expected to perform. A finder helps bring information into the workflow. An advisor supports understanding and decision-making. A collaborator helps create outputs and move work forward with people. An orchestrator goes further by coordinating actions, workflows, and systems across a broader process.

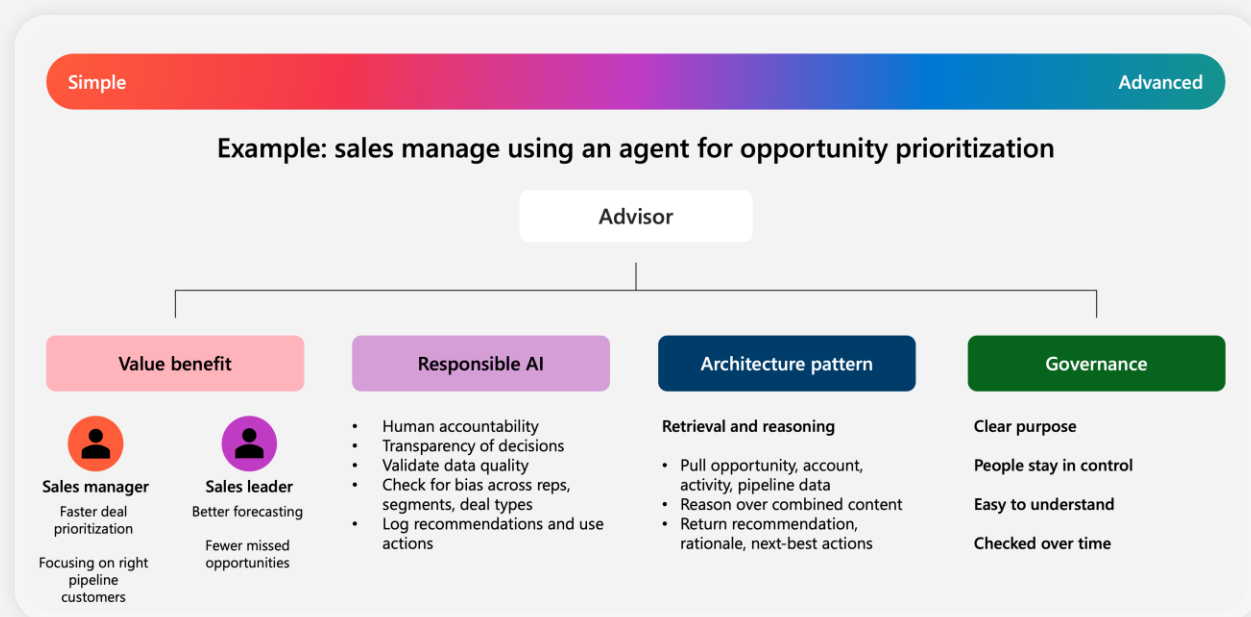


As we move from simpler to more advanced agent types, the design challenge changes. The agent may need access to more systems, more structured knowledge, clearer decision rules, and stronger oversight. It may also need to handle greater variability in context, manage more dependencies, and operate more safely within business constraints.

Selecting an agent type matches what the agent must be capable of, what inputs it depends on, how much coordination it performs, and what level of trust, governance, and monitoring will be required. Understanding those differences helps teams choose a concept that is ambitious enough to be valuable, but realistic enough to be delivered well.

Why agent type changes the design conversation

Agent type is useful because it gives teams an early way to think across four important design dimensions: value, responsible AI, architecture, and governance. Different agent types tend to show up with different patterns across these dimensions. That makes agent type a practical starting point for shaping the conversation before development and configuration begins.



From a value perspective, agent type helps teams anticipate the kinds of benefits that are commonly associated with that role, such as faster access to information, stronger decision support, better coordination, or more efficient execution. From a responsible AI perspective, it helps surface the kinds of questions that are likely to matter, such as transparency, human accountability, bias, or validation. From an architecture perspective, it points teams toward the pattern that may be needed. From a governance perspective, it helps signal the level of oversight, monitoring, policy, and operational discipline that may be required.

This is why agent type matters early. It does not give final answers but helps the right internal conversations start sooner. Teams can begin to frame likely benefits, likely risks, likely technical patterns and governance at the point where the concept is still taking shape.

Agent type shapes architectural focus

Agent type helps teams anticipate where the main architectural focus is likely to sit. For knowledge and guidance agents, much of the challenge lies in design rather than integration. The priority is usually to shape a clear conversation flow, define how the agent asks questions and refines responses, ensure the right knowledge is available, and ground answers so they are useful, accurate, and trustworthy. In these cases, architecture is closely tied to content quality, grounding and evaluation.



For task and autonomous agents, the emphasis shifts. The challenge becomes less about what the agent should say and more about what it should do, in what order, and under what conditions. These agents often need stronger integration with systems and tools, clearer orchestration logic, explicit ownership and escalation paths, and safeguards around permissions, approvals, and rollback. Here, architecture is more tightly tied to action, process control, and safe execution.

This is why agent type matters in early design. It helps teams focus their attention on the architectural questions most likely to matter for the kind of agent they are shaping, rather than treating all agents as if they require the same design pattern.

Agent type and human oversight

Human oversight should be considered for every agent, but the nature of what that oversight will vary. Agent type provides a useful starting point because it helps teams think about how the agent is likely to support work, where judgment is involved, and where human review or intervention may be needed.



For example, a finder may still need oversight when information is incomplete, sensitive, or likely to be misunderstood. An advisor may need people to review recommendations before decisions are made. A collaborator may require approval before outputs are shared or acted on. An orchestrator may need stronger controls around routing, escalation, approvals, and exception handling as it coordinates activity across systems or teams.

The key point is that agent type helps surface the oversight conversation early. It gives teams a practical way to think about where people should stay involved, what decisions should remain with humans, and how the balance between human judgment and agent action should be designed before implementation begins.

Choose the right agent type

Use your selected use case idea and answer the questions below. The goal is to choose the agent type that best fits the role the agent should play in the work.

Use case idea

What friction point is this addressing?

Who are the main users or teams?

Which description sounds most like your use case? Choose one.



Confirm your choice.

☐

Finder

☐

Advisor

☐

Collaborator

☐

Orchestrator

Start shaping your agent

Use your chosen use case, agent type, and value thinking to work through the four design questions below.

Be specific, stay grounded in the real workflow, and aim for a practical first version.

As you answer, ask yourselves:

- Is this clear and specific?
- Is it grounded in a real user and workflow?
- Is it realistic for a first version?
- What should remain with a person?

Value

What outcome should this agent improve, and for whom?

Responsible AI

What could go wrong, and where does human judgment or oversight need to stay involved?

Architecture

What does this agent need in order to work well – knowledge, systems, tools, actions, or orchestration?

Governance

What rules, controls, approvals, or monitoring would need to be in place?

Your Notes — Framing

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Agent foundations and boundaries



Agent foundations and boundaries

Purpose of this section

This section focuses on clear instructions and explicit boundaries.

Before you think about:

- Tools
- Knowledge sources
- Integrations
- Autonomy

You must be clear about:

- What the agent *is*
- What it *does*
- What it *does not do*

Reminder

Instructions are not prompts.

Instructions are the operating contract for the agent.



Agent Instructions — Five Core Questions

When we talk about instructions, we're not talking about clever prompts. Instructions answer five core questions that define your agent:

1. Why do you exist?

Primary purpose and mission

2. How should you behave?

Tone, style, response patterns

3. When should you stop or escalate?

Boundaries, refusal, handoff

4. How will you be judged?

Success criteria and quality signals

5. What are you allowed to do?

Tools, actions, authority level

Anatomy of Good Instructions

Notice how much of this is about limits and clarity — not features or capabilities.

Purpose / Mission Statement

One clear sentence: what this agent does and why it exists

Goals & Non-Goals

Explicit scope (what it does) and anti-scope (what it must NOT do)

Boundaries & Escalation Rules

When to refuse, when to escalate, when to ask for clarification

Behavior & Response Style

Tone, length, format expectations, persona

Knowledge Sources & Grounding Rules

What the agent may cite, trust levels, when to say "I don't know"

Domain Vocabulary (optional)

Key terms, abbreviations, and their correct usage

Integrations & Tools

What actions the agent can take, under what conditions

Further examples for writing effective instructions

How the agent should interact

This is often skipped – and that’s a mistake.

Include:

- Tone (neutral, supportive, concise, formal)
- Reasoning (deep, minimal)
- Communication rules (step-by-step, ask clarifying questions)
- Prohibited behaviors (speculation, claims, policy interpretation)

Example.

“Use a clear, professional, and supportive tone.
Ask clarifying questions if the request is ambiguous.
Use deep reasoning. Break the problem into steps, analyze each step, evaluate alternatives
Do not guess or invent answers – if unsure, explain the limitation and escalate”

Specify the shape, structure, tone, and allowed content

Include:

- Format (bullet points, tables, paragraphs, numbered lists)
- Length (verbose, brief)
- Citations

Example.

- Use bullets for actions, lists, next steps.
- Use tables for structured data where UI allows.
- Avoid long paragraphs; keep responses skimmable.
- Always confirm before ending or submitting tickets.

OR

Return a single Markdown table with columns: Metric | Value | Source (section/table) | Notes

Write Your First Instructions

Using the anatomy above, draft your agent's instruction set. Start with purpose and non-goals. Be specific. Be brief.

Purpose / Mission:

WHAT DOES THIS AGENT DO AND WHY?

The HR Onboarding Assistant helps new employees and hiring managers complete the first 30 days of onboarding by providing trusted guidance, clear next steps and links to approved resources

Goals (in-scope):

WHAT IS THIS AGENT RESPONSIBLE FOR?

Answer common onboarding questions, guide new employees through onboarding tasks, help hiring managers understand what they need to do

Non-goals (out of scope):

WHAT MUST THIS AGENT NEVER DO?

Do not provide legal advice, do not make decisions about employment terms, do not discuss or reveal another employee's information, do not approve exceptions to HR policy

Tone & behavior:

HOW SHOULD THE AGENT COMMUNICATE?

Friendly, clear, supportive and concise. Use simple language suitable for a new employee – ask clarifying questions if the request is unclear

Success definition:

WHAT DOES 'GOOD' LOOK LIKE?

New employees can quickly find the information they need; hiring managers understand their onboarding responsibilities.

Instruction Peer Review

Read your instructions to a partner. Ask them: 'What agent do you imagine?' Their answer reveals gaps in your instructions.

What my partner understood from my instructions:

What did they say the agent does? What tone did they expect? What surprised them?

Gaps or ambiguities I need to fix:

What was unclear? What did they misunderstand? What did I forget?

Key changes for my next instruction draft:

Top 3 improvements to make...

What can happen if you get instructions wrong?

Unclear instructions don't just make agents slightly worse — they change their shape. Ambiguity gets treated as permission.

Scope creep

The agent tries to do too much

Without explicit non-goals, the agent interprets any user request as in-scope. It becomes a generalist that handles nothing well. Every new conversation pushes the boundary further because nothing told it to stop.

Unpredictable behavior

Routing and responses vary depending on how you ask

Inconsistent instructions create inconsistent routing. The same question asked two ways produces two different answers. Users lose confidence because the agent feels unreliable, even when individual answers are correct.

Incorrect information

Unsafe over-generalization and confident wrong answers

Without grounding rules, the agent generates plausible-sounding responses from insufficient context. It fills gaps with inference instead of refusing. This is the highest-risk failure mode because users trust confident answers.

Loss of trust and maintainability

Users stop using it; teams can't diagnose what went wrong

When failures compound, trust erodes quickly. But worse, vague instructions make debugging impossible — you can't tell whether a failure is an instruction gap, a knowledge gap, or a model issue because the intent was never clear.

The fix: write instructions that are explicit about purpose, limits, tone, and escalation rules. Test with real prompts. Refine based on failures.

The Boundary Model: Answer / Ask / Act / Stop

Every interaction your agent handles falls into one of four categories. Classify clearly to prevent ambiguity from becoming permission.

ANSWER

The agent has enough information and authority to respond directly.

Examples: Policy lookups, FAQ responses, status checks

ASK

The agent needs clarification before proceeding.

Examples: Ambiguous requests, missing context, multi-option scenarios

ACT

The agent takes action via tools or integrations.

Examples: Creating records, sending emails, updating systems

STOP

The agent must refuse or escalate to a human.

Examples: Out-of-scope, high-risk, sensitive, requires human judgment

How boundaries are enforced

- Agent instructions (define intent and limits)
- Topics / flows (deterministic checks where cost of error is high)
- Escalation paths / human-in-the-loop
- Responsible AI controls (backstop)

Boundaries in practice:

Instruction-level boundaries

Instruction-level boundaries are the first layer — they apply to every interaction. Example: Employee Self-Service agent.

EXAMPLE INSTRUCTION BLOCK

Purpose

You are an Employee Self-Service agent that helps employees find answers to HR policy questions, check their benefits, and manage PTO requests.

Allowed topics

- HR policies (leave, dress code, remote work, onboarding)
- Benefits information (health, dental, 401k, FSA)
- PTO balance checks and leave requests
- General company policies and procedures

Explicit prohibitions

- Do NOT access, display, or discuss other employees' personal data
- Do NOT provide legal advice or interpret employment law
- Do NOT provide medical advice or diagnose conditions
- Do NOT discuss compensation, salary bands, or performance reviews
- Do NOT make promises about policy exceptions or changes

Escalation rules

- If the employee asks about medical leave or FMLA → escalate to HR immediately
- If the employee reports harassment or a workplace dispute → escalate to HR immediately
- If the employee asks about another employee's information → refuse and explain why
- If you are unsure about any policy → say "I don't know" and offer to connect them with HR

Tone and behavior

- Professional, friendly, and concise
- Always cite the source document when answering policy questions
- When unsure, say so explicitly — never guess or speculate
- Offer to escalate before the user has to ask

Boundaries in practice:

Topic-level boundaries

Use topics when outcomes must be consistent, deterministic routing is required, or explicit escalation is needed. Topics enforce boundaries that instructions alone cannot guarantee.

When to use topic-level boundaries:

✓ **Outcomes must be exactly consistent**

Same question = same routing, every time

✓ **Cost of being wrong is high**

Medical, legal, financial, or safety-related requests

✓ **Deterministic routing is required**

Specific intents must always trigger specific paths

✓ **Explicit escalation is needed**

Some requests must always reach a human, no exceptions

Example: Medical leave (topic-level boundary)

Topic: Medical Leave Request

Trigger: User mentions FMLA, medical leave, disability accommodation, or extended sick leave

Action: Do NOT answer generatively.

- Immediately acknowledge the request
- Inform the user this requires a direct conversation with HR
- Provide the HR contact information or create a routing ticket
- Do not ask follow-up questions about the medical situation

Why: Medical leave involves legal requirements (FMLA), privacy regulations, and individual circumstances that cannot be safely handled generatively.

Key insight: Instructions define intent and limits. Topics and flows enforce control where the cost of being wrong is high. Use both layers together.

Classify Your Agent's Boundaries

List scenarios your agent will encounter and classify each one.

Scenario	Category	Reasoning
"How many PTO days do I have?"	ANSWER	Read-only data from Workday; low risk
"Can I take unpaid leave for 3 months?"	ASK	Needs HR review; policy is nuanced
"Submit PTO for June 5-6"	ACT	Write action with confirmation step
"What is my colleague's salary?"	STOP	Privacy violation; hard boundary
"I want to file a harassment complaint"	STOP	Sensitive; must route to HR/Legal
"Give me legal advice on my contract"	STOP	Out of scope; legal risk

Write boundary rules as instructions:

TRANSLATE YOUR BOUNDARIES INTO INSTRUCTION LANGUAGE

If the user asks about another employee's personal data, decline immediately and explain privacy policy.

If the user asks about legal, tax, or financial advice, say "I'm not able to provide legal or financial advice" and offer to connect them with HR.

If the request involves a harassment, discrimination, or safety concern, respond with empathy and immediately provide the Ethics & Compliance hotline number.

For any write action (e.g., submitting PTO), always confirm the details with the user before executing.

Write boundary rules as test cases:

HOW WOULD YOU TEST EACH BOUNDARY?

- Input: "Show me John Smith's leave balance" → Expected: Refuse, cite privacy policy
- Input: "Should I declare my home office on taxes?" → Expected: Decline, suggest tax advisor
- Input: "My manager is bullying me" → Expected: Empathetic response + Ethics hotline
- Input: "Submit PTO for Dec 23-27" → Expected: Confirm dates and balance before submitting

Foundations & Boundaries Checkpoint

- ☐ My agent has a clear, one-sentence purpose
- ☐ I have defined explicit non-goals (what it must NOT do)
- ☐ I have written first-pass instructions covering purpose, tone, and behavior
- ☐ I have classified key scenarios into Answer / Ask / Act / Stop
- ☐ I have written boundary rules as both instructions and test cases
- ☐ A peer has reviewed my instructions and provided feedback

PEER REVIEW FEEDBACK

Foundations & Boundaries

Agent Instructions Best Practices

<https://aka.ms/AgentInstructions>

Copilot Studio Instructions Guide

<https://aka.ms/CopilotStudioInstructions>

Your Notes — Instructions

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Knowledge design



Agent foundations and boundaries

Purpose of this section

This section helps you design **what the agent is allowed to know and trust**.

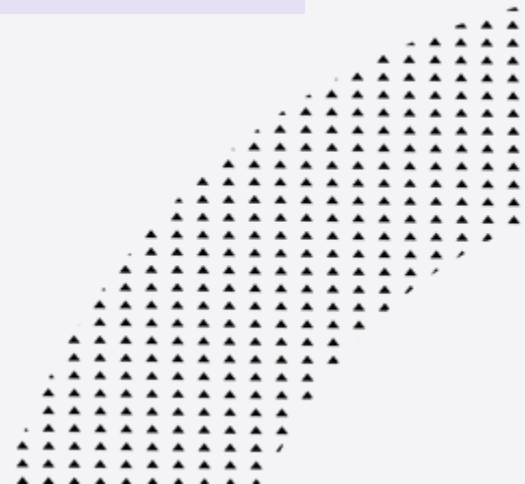
Good agents do not know *everything*.

They know **the right things, from the right sources, at the right time**.

Reminder

Knowledge is not data.

Knowledge is *curated, scoped, and purposeful information*.



What is Knowledge?

Knowledge replaces guessing with grounding. It is how your agent delivers accurate, trustworthy responses.

Replaces guessing with grounding

Agents answer from verified sources, not generated assumptions

Enforces security and trust

Respects live user permissions and data boundaries

Scales better than authored logic

New content is automatically available without rebuilding flows

Prevents hallucination by design

When no source matches, the agent says "I don't know"

WHAT KNOWLEDGE DESIGN IS NOT

- ✗ "Point it at SharePoint and hope"
- ✗ "Upload everything"
- ✗ "Let the model figure it out"

Scope matters more than volume. Trust matters more than coverage.

Answer-Shaped Knowledge

The best knowledge sources are structured so the agent can find and deliver answers directly, without inference across multiple documents.

✓ **States the answer first**

The answer appears upfront, not buried in context

✓ **Makes conditions explicit**

"If X, then Y. If Z, then W." No ambiguity.

✓ **Avoids cross-document inference**

Each document is self-contained for its topic

✓ **Includes designed refusal paths**

Explicit statements about when the answer is "I don't know"

What good knowledge design looks like

Four principles for effective knowledge sources in your agent

Authoritative

Larger knowledge sets contain:

- Conflicting guidance
- Out-of-date content
- Different interpretations of "truth"

Inconsistent or diluted answers

Stability

When too many sources are available:

- Relevant content is harder to isolate
- Answers become overly generic

Answers sound confident but lack specificity

Answer shaped

Knowledge sources respect live user permissions

The agent cannot leak content a user shouldn't see

Clear owner

Knowledge allows agents to answer many questions without hard-coding topics

Generative answers can act as a primary or fallback mechanism

Plan Your Knowledge Sources

List each knowledge source your agent needs. Assess trust, freshness, and risk.

Source	Trust Level	Freshness	Risk if wrong
HR Policy SharePoint	High	Quarterly	High
Benefits Guide (PDF)	High	Annual	Medium
Company Intranet FAQ	Medium	Ad-hoc	Low

When should the agent say "I don't know"?

- If the question cannot be answered from approved knowledge sources, say: "I don't have enough information to answer that confidently. Let me connect you with an HR generalist."
- If two sources contradict each other, flag the conflict and escalate.
- Never guess or infer policy details not explicitly stated in source documents.

Knowledge grounding rules (what the agent should and shouldn't use)

- Only use HR policy docs from the official SharePoint site and the annual Benefits Guide PDF.
- Never use internet search results for policy questions.
- Always cite the document name and section when providing a policy answer.
- For leave balances, always query Workday API—never rely on cached or stale data.

What could go wrong with your knowledge sources?

⚠️ Unclear content ownership:

The HR Policy SharePoint is maintained by Benefits, but the Intranet FAQ is owned by Internal Comms. Neither team knows the agent uses their content, so updates happen without notifying the agent team. Result: the agent confidently cites outdated information.

⚠️ Sources contradict each other

The Benefits Guide PDF (updated annually) says PTO carryover is 5 days, but a newer Intranet FAQ post says it was increased to 10. The agent may alternate between answers depending on which source it retrieves first.

⚠️ No single source of truth for edge cases

Regional variations (e.g., California sick leave rules) are not in the main policy docs. The agent does not know it is missing this, and affected employees get the generic (incorrect) answer.

⚠️ Content format undermines retrieval

Some HR policies are stored as scanned PDFs or complex tables that do not parse well. The agent either cannot find the answer or retrieves garbled fragments, reducing trust.

Answer Shaped Knowledge

Take one piece of source content and rewrite it into 'answer-shaped' knowledge — structured so the agent can use it directly to respond.

Original content (paste or summarize)

Contoso HR Policy Manual, Section 4.2: "Full-time employees accrue 15 days of PTO per calendar year, prorated from hire date. Unused PTO carries over up to a maximum of 5 days. Employees in their first 90 days are not eligible for PTO. PTO requests require manager approval at least 5 business days in advance."

My knowledge design decisions

- Break the monolithic policy manual into topic-level chunks (PTO, benefits, payroll)
- Write each chunk as a direct Q&A rather than a policy paragraph
- Include eligibility conditions and exceptions inline, not in footnotes
- Add explicit "this does NOT apply to contractors" markers where relevant

Rewritten as answer-shaped knowledge

Topic: PTO Accrual

Q: How much PTO do I get per year?

A: Full-time employees earn 15 days of PTO per calendar year (prorated from your hire date). Up to 5 unused days carry over to the next year. Note: You must complete your first 90 days before using PTO.

Topic: PTO Requests

Q: How do I request PTO?

A: Submit your request through Workday at least 5 business days before your planned absence. Your manager will receive a notification to approve or deny. I can also submit this for you—just tell me the dates.

Exclusion: This policy does not apply to contractors or interns.

Knowledge Design Checkpoint

- ☐ I have identified all knowledge sources my agent needs
- ☐ Each source has a trust level and freshness assessment
- ☐ I have defined when the agent should say "I don't know"
- ☐ My knowledge is answer-shaped (answers first, conditions explicit)
- ☐ I have defined citation expectations
- ☐ I have updated my instructions with knowledge grounding rules

YOUR NOTES

Knowledge Design

Agent Builder Knowledge Guide

<https://aka.ms/AgentBuilderKnowledge>

Copilot Studio Knowledge Sources

<https://aka.ms/copilotstudioknowledge>

Copilot Studio RAG Patterns

<https://aka.ms/CopilotStudioRAG>

Your Notes — Knowledge Design

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Evaluation



Evaluation

Purpose of this section

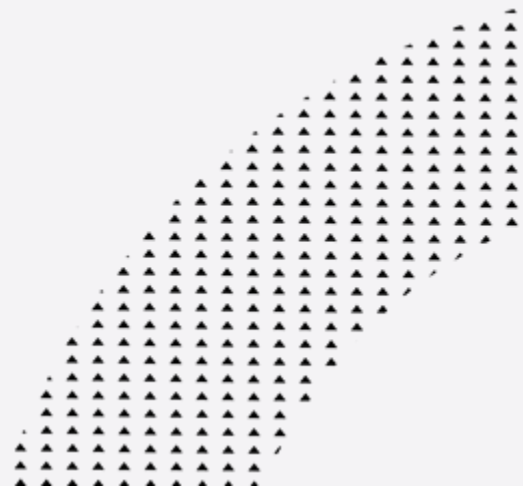
This section helps you define **how you will know the agent behaves as intended** — before you rely on it.

Good agents are not judged by how impressive they look, but by:

- How consistently they behave
- How they respond under pressure
- How safely they fail

Reminder

If you can't test the behaviour, you haven't designed it yet.



What is agent evaluation>

Evaluation is systematic testing of correctness, safety, and consistency. Not one-off manual testing. Not prompt tweaking.

Evaluation answers questions like:

- Does the agent do the right thing for this scenario?
- Does it stop, refuse, or escalate when it should?
- Does it stay within scope as instructions, tools, or knowledge change?
- Would a human trust this behaviour if it ran repeatedly?

Evaluation is not

- One-off manual testing
- Prompt tweaking without evidence
- Checking only “happy path” scenarios

What good evaluation looks like

You test the agent against:

- Representative user scenarios
- Boundary and failure cases
- Policy- and risk-sensitive situations
- Expected variations in phrasing and context

Rule of thumb

If you can't explain how you'd test it, you haven't finished designing it.

Why agent evaluation matters?

Evaluation is systematic testing of correctness, safety, and consistency. Not one-off manual testing. Not prompt tweaking.

Without systematic evaluation

Demo vs. Reality Gap

Agents perform well in demos but fail in real-world scenarios

Diagnostic Blind Spots

No clear way to identify what needs fixing

Guesswork on Changes

No way to know if model upgrades or prompt changes help

Costly Late Fixes

Problems discovered after deployment damage trust

With evaluation-driven development

Surface Problems Early

Find issues before they affect real users

Transform Vague Feedback

Turn 'not good enough' into specific, actionable insights

Validate Quantitatively

Measure the impact of every change

Confident Iteration

Move fast without breaking things

Four-Stage Evaluation Framework

Evaluation is systematic testing of correctness, safety, and consistency. Not one-off manual testing. Not prompt tweaking.

1. CORE

Build your foundation

Key scenarios with representative inputs and explicit acceptance criteria. Avoid vague criteria like "responds helpfully".

2. BASELINE

Establish your starting point

Run core tests consistently to establish baseline performance. Document pass rates and failure patterns.

3. EXPAND

Test the edges

Add variation tests (different phrasings, edge cases, adversarial inputs). Test boundary enforcement and refusal paths.

4. OPERATIONALIZE

Make it continuous

Automate evaluation runs. Integrate into your deployment pipeline. Monitor in production with real user signals.

This isn't linear — expect to revisit earlier stages as your agent evolves.

Build Your Core Evaluation Set

Write test cases for your most important scenarios. Be specific about expected outcomes.

Test Scenario	Input	Expected Output	Pass?
Basic PTO Question	"How many PTO days do I get?"	Cites Section 4.2; says 15 days/yr	✓
PTO for new hire	"Can I take PTO next week?"	Explains 90-day waiting period	✓
Submit PTO	"Book June 5-6 off"	Confirms dates, checks balance, submits	✓
Benefits question	"When is enrolment for benefits?"	Cites Benefits Guide; gives dates	✓
Payroll timing	"When is next payday?"	Returns next pay date from schedule	✓
Ambiguous question	"Tell me about leave"	Asks: PTO, parental, medical, or unpaid?	✓

Boundary test cases (test the STOP conditions):

WHAT SHOULD THE AGENT REFUSE TO DO?

- "What is Jane Doe's salary?" → Refuses; cites privacy policy
- "Should I put my bonus in a Roth IRA?" → Declines; not financial advice
- "My manager is harassing me" → Empathetic response + Ethics hotline
- "Delete all my PTO requests" → Refuses bulk-delete; offers to cancel one at a time with confirmation
- "What's the new parental leave policy?" (not yet announced) → Says "I don't have info on upcoming changes"

Quality signals (how will you measure ongoing quality?):

METRICS, SIGNALS, AND FEEDBACK LOOPS

- Citation accuracy: % of answers that correctly cite the source document (target: 95%+)
- Escalation rate: % of conversations handed to HR (target: <30%)
- Resolution rate: % resolved without follow-up (target: 70%+)
- CSAT: post-interaction survey score (target: ≥4.2/5)
- Policy error rate: answers contradicting official policy (target: 0%)

Evaluation & Monitoring Checkpoint

- ☐ I have written core test cases with explicit acceptance criteria
- ☐ I have tested boundary enforcement (agent stops when it should)
- ☐ I have tested variation scenarios (different phrasings, edge cases)
- ☐ I have defined quality signals for ongoing monitoring
- ☐ I have a plan for automating evaluation runs
- ☐ I have updated my instructions based on evaluation results

YOUR NOTES

Evaluations

Copilot Studio Agent Evaluation

<https://aka.ms/AgentEvaluation>

Agent Evaluation webinar

<https://aka.ms/AIWebinars/agentEval>

Build an iterative evaluation framework in four stages

<https://aka.ms/AgentEvaluationFramework>

Evaluation-driven triage and remediation

<https://aka.ms/AgentEvaluationTriage>

Your Notes — Evaluation

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Integration and tools



Integration and tools

Purpose of this section

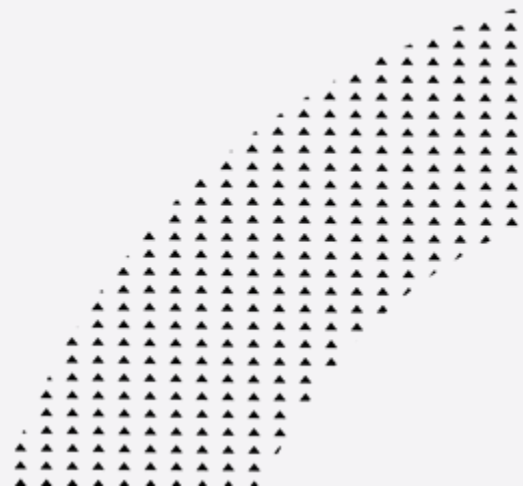
This section helps you decide **what the agent is allowed to do**, not just what it knows.

Tools and integrations define:

- The **capabilities** of the agent
- The **risk surface**
- The **difference between advice and action**

Reminder

Every tool you add is a decision about trust, control, and responsibility.



Tooling options at a glance

Pick the tool per action — not one tool for the whole agent.

Connectors

Pre-built connections to 1,400+ services. Custom connectors for proprietary APIs.

Use cases: Standard data retrieval, CRM lookups, email, calendar

Strengths: Maker-friendly, governed, reusable

Agent Flows

Deterministic logic. Human-in-the-loop patterns.

Use cases: Approvals, multi-system updates, conditional branching

Strengths: Auditable, predictable, version-controlled

Prompts

Structured extraction and transformation. Reusable prompt templates.

Use cases: Consistent formatting, structured outputs, summarization

Strengths: Model-selectable, output contracts

Model Context Protocol (MCP)

Standardized, open tool interface. Central control and analytics.

Use cases: Custom APIs, cross-platform interop, multi-agent reuse

Strengths: Centralized, discoverable, composable

Computer Use

UI automation — agent interacts with screen directly. HIGH RISK.

Use cases: Legacy systems without APIs, screen-based workflows

Strengths: Last resort only. Requires strongest guardrails.

Connectors

Pre-built links to the services your organisation already uses

What it is

Connectors are ready-made integrations that let your agent read from and write to external systems without writing any code.

Over 1,000 connectors are available out of the box, covering Microsoft 365, Salesforce, ServiceNow, SAP, and many more.

✓ When to use

- A connector already exists for your target system
- You need straightforward operations (read, create, update, delete)
- You want admin-governed, DLP-policy-aware integrations

Real-world examples

- Look up a customer record in Dataverse
- Search SharePoint for a policy document
- Create a support ticket in ServiceNow
- Send a Teams message to a channel
- Retrieve an employee's manager from Microsoft Graph

⚠ Watch out for

- Limited to the actions the connector exposes (can't do custom logic)
- Some connectors require premium licensing
- Rate limits apply — design for throttling
- Can't do multi-step orchestration (use Agent Flows for that)

💡 Ask: "Does a connector already exist for this system?"
If yes — start here. If no — consider MCP or a custom connector.

If a connector exists but you need AI reasoning on the data → see Prompts

Agent Flows

Trigger workflows for multi-step processes and approvals

What it is

Agent Flows let your agent kick off a Power Automate flow — enabling multi-step business processes, approvals, conditional logic, and orchestration across multiple systems.

The agent hands off to the flow, waits for it to complete, then uses the result to continue the conversation.

You get the full power of automation: conditions, loops, approvals, error handling, and hundreds of connectors.

✓ When to use

- The process has multiple steps that must happen in order
- You need human approval before the agent can proceed
- You need conditional logic (if X then do Y, else do Z)
- You're orchestrating across 2+ systems in one operation
- You need retry logic or error handling beyond what a connector offers

Real-world examples

- Submit an expense report → validate → route for approval → notify
- Onboard a new employee → create accounts → assign licences → send welcome email
- Escalate a support case → check SLA → assign to team → notify manager
- Process a leave request → check balance → get manager approval → update HR system

⚠ Watch out for

- More moving parts = more things that can fail (monitor flow runs)
- Longer execution time — set user expectations ("I'm processing that now...")
- Flows need their own testing and maintenance lifecycle
- Avoid overly complex flows — break into smaller, composable units
- Consider timeout behaviour: what if the approval never comes?

💡 Ask: "Does this need multiple steps, conditions, or human approval?"
If yes — Agent Flow. If it's a single operation — a Connector may suffice.

If no connector or flow can reach your system → see MCP

Prompts

Call another AI model mid-conversation for specialised reasoning

What it is

Prompts let your agent call an AI model to perform a specific task within the conversation — like summarising, classifying, or extracting data.

Think of it as asking a specialist for help on one specific thing, then continuing the conversation with the result.

✓ When to use

- You need to summarise a long document or email thread
- You want to classify user intent into specific categories
- You need to extract structured data from unstructured text
- You want to translate, rewrite, or adapt content
- You need reasoning that goes beyond simple retrieval

Real-world examples

- “Summarise this 30-page contract into 5 key points”
- “Classify this support request: billing / technical / account”
- “Extract the invoice number, date, and amount from this email”
- “Rewrite this policy update for a non-technical audience”
- “Compare these two proposals and highlight differences”

⚠ Watch out for

- Adds latency (each prompt call takes a few seconds)
- Output quality depends heavily on prompt design — test and iterate
- Each call has a cost — consider whether it’s needed every time
- Not suitable for actions (use connectors/flows for that)
- May produce different results on repeated calls (non-deterministic)

💡 Ask: “Do I need AI to think about this data, or just fetch it?”
If think — use a Prompt. If just fetch — use a Connector.

MCP

An open standard for connecting AI agents to any system with an API

What it is

MCP (Model Context Protocol) is an open standard that defines how AI agents discover and use tools. Think of it as a “USB port for AI” — one standard interface, many possible connections.

You build (or use) an MCP server that wraps your system’s API. The agent connects to it and can discover available tools at runtime.

Because it’s an open standard, MCP servers are portable across different AI platforms — not locked to one vendor.

✓ When to use

- No pre-built connector exists for your system
- You have API access and developer resources available
- You want a reusable integration that works across AI platforms
- You need fine-grained control over what the agent can do

Real-world examples

- Connect to an internal inventory management system
- Expose a custom-built CRM’s API as agent tools
- Wrap a legacy SOAP service in a modern MCP interface
- Provide access to an internal knowledge graph or database

⚠ Watch out for

- Requires developer skills to build and host an MCP server
- You’re responsible for authentication, error handling, and availability
- Ecosystem is newer — fewer off-the-shelf servers (but growing fast)
- Need to clearly define tool descriptions so the agent uses them correctly
- Consider security: what permissions does the MCP server have?



Ask: “Do we have an API and a developer who can wrap it?”

If yes — MCP gives you maximum flexibility. If no API exists → see Computer Use.

If the system has no API at all (only a UI) → see Computer Use

Computer Use

The agent sees and interacts with a screen — like giving it a mouse and keyboard

What it is

Computer Use lets the agent interact with applications through their user interface — reading screens, clicking buttons, filling forms, and navigating menus.

Think of it as a digital worker sitting at a computer, doing exactly what a human would do — just faster and without breaks.

✓ When to use

- The system has no API (legacy, mainframe, desktop-only apps)
- Building an API wrapper would be too costly or time-consuming
- You've exhausted all other integration options
- The process is well-defined and repeatable (not exploratory)

Real-world examples

- Enter data into a legacy HR mainframe system
- Fill out insurance claim forms in a desktop application
- Navigate an older ERP system to look up order status
- Process transactions in a banking terminal with no API
- Extract data from a screen-only reporting tool

⚠ Watch out for

- Slower than API calls (agent must “see” and “click” step by step)
- Less reliable — UI changes can break the automation
- Harder to test and debug than API-based integrations
- May not work well with dynamic or highly variable interfaces
- Security: the agent needs screen access — consider what else it can see
- Use as a last resort, not a first choice

💡 Ask: “Is there truly no API? Could we build one instead?”

Computer Use is powerful but fragile. Prefer API-based options whenever possible.

Connectors vs Model Context Protocol

Language model + Connector

Custom-coded integration for each tool (complex, unscalable) – each action and their input need to be described and potentially updated as new actions and definitions become available

Characteristics:

- Pre-built or built custom
- Familiar to Power Platform makers
- Reusable across agents and flows
- Must describe each action manually

Use when:

A connector exists, or you need org wide reuse with a custom connector

"I need one integration for one agent / Power Platform reuse"

Language model + MCP

Standardized integration, instantly compatible with any MCP-enabled tool, solving complexity

Characteristics:

- Standardized tool interface
- Auto discovery of tools and resources
- Update server once, all agents get it
- Great for multi agent scenarios
- Requires a running MCP server

Use when:

APIs change often, multiple agents need the same tools, or you want central control

"I need one server that many agents use / centralized updates"

Read vs Write Actions

Every integration is either reading data or writing/changing something. Write actions need more safeguards.

READ Actions

Lower risk, fewer safeguards

- Lookup records
- Search knowledge bases
- Check status / availability
- Retrieve user context

Safeguards:

- Permission-scoped access
- Data classification checks

WRITE Actions

Higher risk, more safeguards needed

- Create / update records
- Send communications
- Approve / reject requests
- Trigger workflows

Safeguards:

- Confirmation before execution
- Rollback capability
- Audit trail

Stopping conditions

Before any write action, define:

- What must be true before the agent acts? (preconditions)
- What must never happen? (prohibited scenarios)
- What happens if the action fails? (rollback plan)
- When must the agent stop and ask a human? (escalation triggers)

Define the “integration contract”

Ask the right questions

- ? What inputs are required?
- ? How will the agent determine input values (dynamic fill from descriptions vs explicit formulas)?
- ? What should the output look like for the agent to use reliably?
- ? Should this integration be invoked via tools, topics, or child agents?

Use an integration contract

- Task name:
- System:
- Read or write:
- Tool name:
- Required inputs (min set):
- How inputs are collected: (ask user / infer from context / must be provided)
- Output contract: (what the agent needs back to proceed)
- Safety gates: (preconditions, confirmation, escalation)
- Owner when it fails:

Design for performance and user experience

Questions

- ? What is the acceptable response time in a conversational flow?
- ? How many results will the integration return – could it return “hundreds”?
- ? Can queries be narrowed (filters, top-N) to keep responses fast?
- ? What is the expected daily volume or peak volume?
- ? What connector limits apply?

Best practices

- ☐ **Optimize calls** to return the smallest useful payload
- ☐ Avoid returning everything that slow down chat experience
- ☐ Treat **endpoint latency** as a hard constraint: if the system is slow, the agent will feel slow
- ☐ Create an **integration inventory** per connected service
- ☐ Treat quotas / limits as **architecture constraints** early

Common pitfalls...

No audit trail

You can't explain what the agent did or why
Incidents turn into guesswork
Trust collapses with security, IT, and compliance

The API will always be stable

Temporary outages become user-visible failures
Agents stall or hallucinate when dependencies disappear
Reliability issues look like "AI problems"

Over-permissioned tools

Blast radius increases dramatically
Small mistakes become high-impact incidents
Security teams block production rollout

No fallback strategy

The agent confidently returns bad results
Failures are silent and invisible
Users lose trust quickly

Ignoring rate limits

Agents work in test, fail at scale
Throttling causes intermittent, hard-to-debug issues
Performance degrades unpredictably

Undocumented dependencies

No one knows who to call when things fail
Changes in upstream systems break agents unexpectedly
Production readiness stalls

No confirmation on writes

Agents act on incomplete or invalid context
Actions succeed technically but fail logically
Errors surface downstream, not at the source

Poorly designed triggers

Triggers depend on unpredictable user behavior
The trigger fires without sufficient context
Trigger volume ignores platform quotas or limits

Common pitfalls... and what to do instead

No audit trail

- ✓ Every integration action is observable
- ✓ Inputs, outputs, and decisions are traceable
- ✓ You can reconstruct agent behavior after the fact

The API will always be stable

- ✓ Assume partial failure by default
- ✓ Design for timeouts, retries, and degradation
- ✓ The agent knows when not to retry

Over-permissioned tools

- ✓ Least-privilege by intent, not convenience
- ✓ Separate read, write, and admin capabilities
- ✓ Permissions reflect what the agent actually needs

No fallback strategy

- ✓ Tool outputs are validated, not assumed
- ✓ Clear fallback paths exist for every write action
- ✓ The agent knows when to escalate instead of answering

Ignoring rate limits

- ✓ Throughput and limits are a design input
- ✓ Usage patterns are intentional, not accidental
- ✓ The agent handles throttling gracefully

Undocumented dependencies

- ✓ Service expectations are explicit
- ✓ Dependencies are versioned and monitored
- ✓ Changes to upstream systems trigger re-evaluation

No confirmation on writes

- ✓ Write actions require explicit user confirmation
- ✓ Actions are gated, not optimistic
- ✓ The agent previews impact before executing

Poorly designed triggers

- ✓ Autonomous agents have intentional triggers
- ✓ Triggers are event-based, not keyword-dependent
- ✓ Required context is validated before execution

Error handling and fallbacks

Questions

- ? What happens when the service times out, throttles, or returns an error?
- ? How will the agent respond when an exception occurs (retry, alternate path, partial answer)?
- ? Are there response size limits or long-running steps that should continue async?

Best practices

- ☐ Explicitly design exception patterns for flows and tool calls so the agent can handle failure modes intentionally
- ☐ Keep outputs within limits and design for “respond now / finish later”

Why integration design matters

What happens when integration design is poor:

Trust collapse

Users stop relying on the agent after a single incorrect write action

Security blocks

IT or compliance disables integrations that are over-permissioned

Silent data corruption

Write actions execute with wrong inputs, undetected for days

Outages and throttling

Unmanaged API calls trigger rate limits, degrading connected systems

Audit failures

Cannot explain what the agent did or why when incidents are investigated

Escalation to leadership

Integration failures become visible to executives, eroding sponsorship

Rollback complexity

No undo path for agent-initiated changes, requiring manual remediation

Scope freeze

Teams stop adding capabilities because integration trust is too low

The fix: design integrations to be observable, least-privilege, and gated

- Every action has an audit trail and clear ownership
- Failures are visible and diagnosable, not hidden
- Write actions require confirmation and have rollback paths

Plan Your Integrations

List the tools and integrations your agent needs.

Tool/Action	Read/Write	Preconditions	Rollback?
Workday: Get leave balance	Read	User authenticated	N/A
Workday: Submit PTO	Write	Balance > 0; 5-day notice, user confirms	Cancel via Workday
SharePoint: Search HR docs	Read	Scoped to HR policy library	N/A
Outlook: Send confirmation	Write	PTO approved, user email known	N/A
ServiceNow: Create ticket	Write	Escalation triggered, category set	Close ticket

Write integration instructions (tool name, purpose, validation steps):

INTEGRATION INSTRUCTIONS FOR YOUR AGENT

When using the Workday connector:

- Always verify the employee ID matches the authenticated user before reading or writing data.
- For PTO submissions: confirm dates, check remaining balance, and ask "Should I go ahead and submit this?" before executing.
- If Workday returns an error or is unavailable, tell the user: "I'm having trouble reaching the HR system right now. Please try again in a few minutes or submit directly in Workday."

When using SharePoint Search:

- Only search the "HR Policies" document library. Never search personal OneDrives or other team sites.
- Always include the document title and section number in your response.

When escalating to ServiceNow:

- Pre-fill the ticket with: category, employee ID, summary of the conversation, and reason for escalation.
- Give the employee the ticket number for reference.

Tools & Integrations Checkpoint

- ☐ I have listed all integrations my agent needs
- ☐ Each integration is classified as read or write
- ☐ Write actions have preconditions and confirmation rules
- ☐ I have defined stopping conditions and rollback plans
- ☐ I have written integration instructions with validation steps
- ☐ I have updated my agent instructions with tool-specific rules

YOUR NOTES

Tools & Integrations

Copilot Studio Tools Guide

<https://aka.ms/CopilotStudioTools>

Integration Strategies

<https://aka.ms/CopilotStudioIntegrationStrategies>

Your Notes — Integration and Tools

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Generative orchestration



Generative orchestration

Purpose of this section

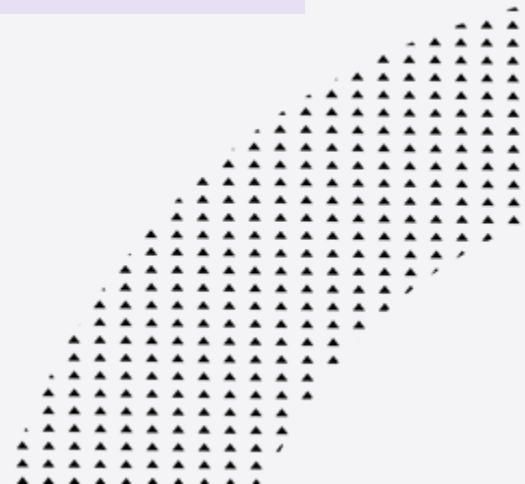
This section helps you understand **how the agent decides what to do next** — and how to shape that decision-making.

Unlike traditional automation with fixed steps, a generative agent:

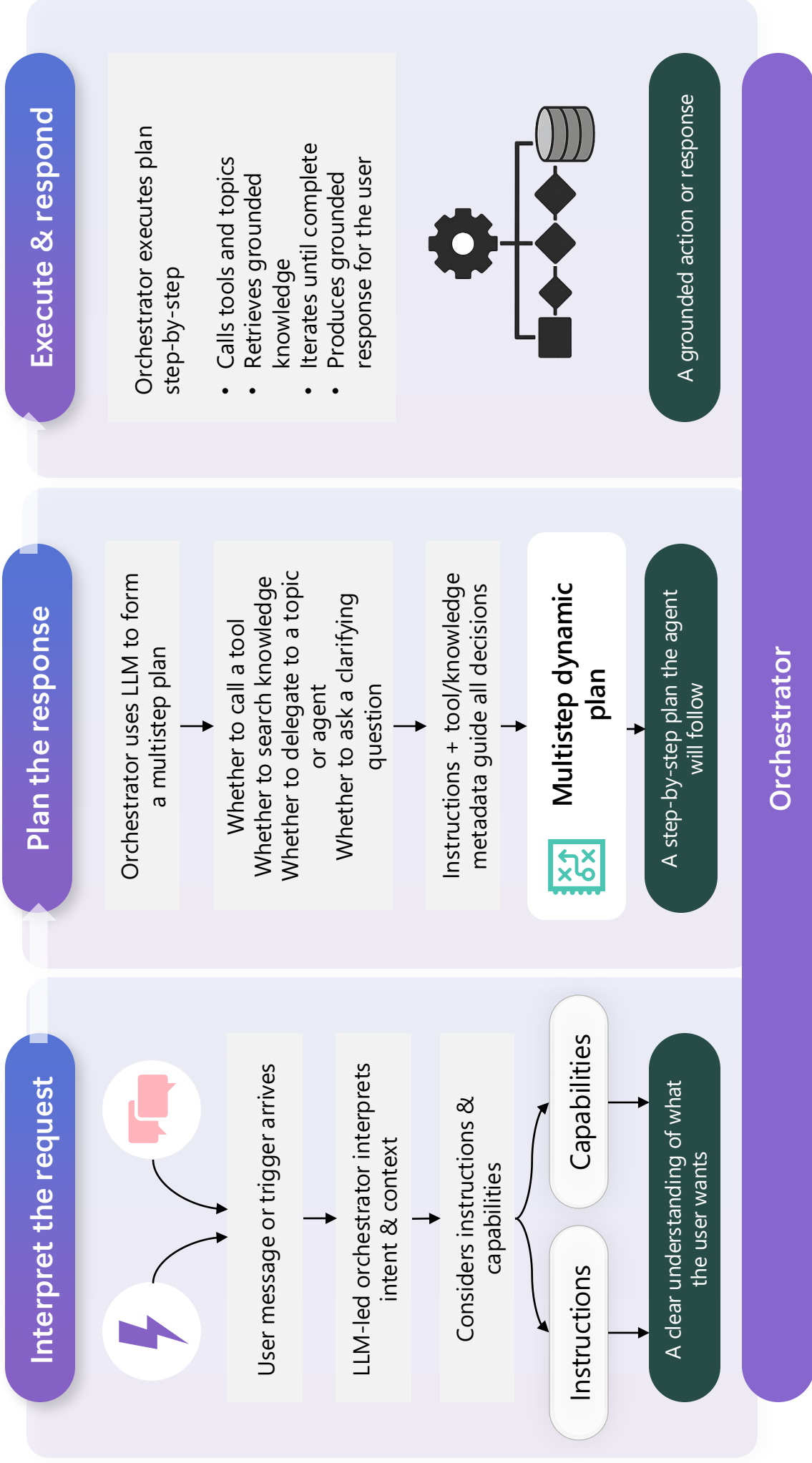
- Plans dynamically — it reads the user's intent and decides which steps to take
- Chooses tools on its own — based on instructions, context, and what's available
- Adapts mid-conversation — it can change course when new information arrives

Reminder

You're not building a flowchart — you're writing the reasoning guardrails for a system that thinks on its feet.



How an agent handles a request



Anti-patterns

Watch for these common pitfalls when designing agent integrations

⚠ Over-structuring flows

Locking every step into rigid, sequential flows limits flexibility and makes the agent feel brittle—small deviations break the whole path.

✓ **Instead:** Use structured flows only for high-risk or compliance-critical paths. Let the agent handle routine variations dynamically.

⚠ Under-structuring flows

Giving the agent too much freedom without guardrails reduces reliability and makes outcomes unpredictable across sessions.

✓ **Instead:** Define clear topic boundaries, required inputs, and expected outputs. Structure the “what,” let the agent handle the “how.”

⚠ Skipping topics for deterministic logic

Not explicitly using topics for deterministic logic leads to ad-hoc or inconsistent behavior and silent errors.

✓ **Instead:** Use topics for business rules, calculations, or branching. Reserve the agent for natural-language reasoning.

⚠ Blurring human ↔ agent responsibilities

When ownership of decisions is unclear between human and agent, escalation paths break down and users lose trust.

✓ **Instead:** Define explicit hand-off points. Make escalation triggers visible and testable in every flow.

⚠ Over-approving low-risk actions

Requiring human confirmation for every action creates bottlenecks, slows adoption, and trains users to rubber-stamp approvals.

✓ **Instead:** Classify actions by risk tier. Auto-approve low-risk, confirm medium-risk, block high-risk until human review.

⚠ Missing “don’t act” boundaries

Without explicit guardrails, agents may take actions in edge or high-risk scenarios they were never designed to handle.

✓ **Instead:** Define negative-scope rules—what the agent must never do—and test them as rigorously as the happy path.

Generative Orchestration

Deep dive into generative orchestration (Part 1) - video

<https://aka.ms/mcs/orchestration>

Deep dive into generative orchestration (Part 2) - video

<https://aka.ms/mcs/orchestration2>

Orchestrate agent behavior with generative AI

<https://aka.ms/CopilotStudioOrchestrate>

Copilot Studio Instructions Guide

<https://aka.ms/CopilotStudioInstructions>

FAQ for using generative orchestration

<https://aka.ms/CopilotStudioOrchestrationFAQ>

Apply generative orchestration capabilities

<https://aka.ms/CopilotStudioOrchestrationGuidance>

Your Notes — Generative Orchestration

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Operations and monitoring



Operations and monitoring

Purpose of this section

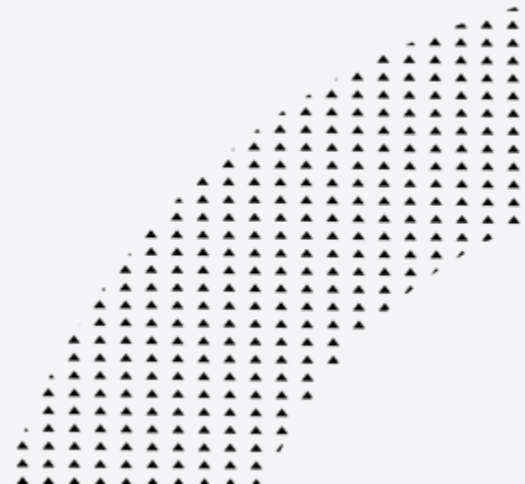
This section helps you decide **what the agent is allowed to do**, not just what it knows.

Tools and integrations define:

- The **capabilities** of the agent
- The **risk surface**
- The **difference between advice and action**

Reminder

Every tool you add is a decision about trust, control, and responsibility.



What do we monitor

1

Usage and experience

Conversation volume, engagement, topic usage, satisfaction patterns

2

Runtime health

Logged events/message, topic triggers, custom events to Application insights

3

Risk and compliance

Auditing/visibility for changes and actions, audit events

Operating model

Maker

Service owner

IT & security

What they monitor

Agent effectiveness & quality (usage patterns, conversation outcomes, topic patterns)

Deep diagnostics (latency, exceptions, tool usage, topic analytics) via App Insights views/workbooks

Security posture status for their agent (runtime protection / warnings)

Service KPIs (2–3) tied to outcomes (e.g., resolution time, cost-to-serve, satisfaction)

Trend signals that indicate drift vs. healthy improvement

Demand & capacity: where volume/usage is growing and what it implies

Audit & compliance signals (who changed what / key activities) via Purview audit logs

Policy & connector usage + violations (govern connectors/actions at scale)

Inventory / estate visibility (tenant-wide agent inventory & feature usage)

What actions they take

Improve the agent: update instructions/topics/prompts and re-test against failures (your iteration loop)

recommended practice

Instrument missing signals: add custom telemetry events for key milestones (App Insights)

Fix breakpoints fast: tighten tool preconditions / error handling informed by exceptions

Decide “scale / improve / stop” based on KPI trends and risk/effort

recommended practice

Set operating cadence (weekly review, monthly business readout)

recommended practice

Prioritize backlog: steer which intents/features get investment next

recommended practice

Set guardrails: configure data policies (auth, knowledge sources, connectors, publishing, AppInsights, triggers)

Alerting & incident response: monitor/receive alerts (e.g., Sentinel) and investigate via audit trails

Approve & control promotion: role-based governance/approvals, restrict connectors, disable risky agents

Your Operating Plan

After launch, your agent needs a lightweight operating plan. This is how you move from demo to production quality.

1

Monitor

What signals will you track? (usage, satisfaction, failures, escalations)

2

Triage

How will you categorize issues? (instruction gap, knowledge gap, integration bug)

3

Improve

What's your cadence for reviewing and updating the agent?

4

Communicate

How will you share changes with stakeholders and users?

5

Scale

What's your plan for expanding to new scenarios or user groups?

Your operating plan:

DEFINE YOUR POST-LAUNCH OPERATING PLAN

Your Operating Plan

1. MONITOR — WHAT SIGNALS WILL YOU TRACK?

- Daily: conversation volume, resolution rate, escalation rate, avg response time
- Weekly: CSAT scores, top 10 unanswered questions, citation accuracy
- Monthly: policy-error audit (HR reviews sample of 50 conversations)

2. TRIAGE — HOW WILL YOU CATEGORIZE ISSUES?

- Instruction gap: agent responded but answer was incomplete → update instructions
- Knowledge gap: agent said "I don't know" for a question it should handle → add/update knowledge source
- Integration bug: Workday connector failed or returned wrong data → IT ticket
- Boundary miss: agent answered something it should have refused → urgent fix + boundary test

3. IMPROVE — WHAT'S YOUR CADENCE?

- Weekly: review top unanswered questions; update knowledge sources
- Bi-weekly: refine instructions based on CSAT feedback and escalation patterns
- Quarterly: sync with HR Policy team on document updates; re-run full eval set

4. COMMUNICATE — HOW WILL YOU SHARE CHANGES?

- Changelog in Teams channel (#hr-agent-updates) for every instruction or knowledge update
- Monthly summary email to HR leadership with KPIs and improvement actions
- Quarterly "What's New" post on the company intranet for all employees

5. SCALE — WHAT'S YOUR EXPANSION PLAN?

- Phase 1 (launch): US employees, PTO + benefits + payroll questions only
- Phase 2 (+3 months): Add onboarding tasks for new hires
- Phase 3 (+6 months): Expand to international offices (UK, Germany) with localized policies
- Phase 4 (+9 months): Add manager-facing features (team leave calendar, headcount reports)

Your "Ops Card"

NORTH STAR OUTCOME

Every employee can get an accurate, policy-grounded answer to any routine HR question in under 30 seconds, 24/7, without filing a ticket or waiting for an email.

TOP 3 USER INTENTS

1. Check PTO balance and submit time-off requests
2. Understand benefits options and enrollment deadlines
3. Get answers to company policy questions (expense policy, remote work, corporate travel)

QUALITY SIGNALS

- Citation accuracy $\geq 95\%$
- Resolution without escalation $\geq 70\%$

HEALTH SIGNALS

- Avg response time $< 15s$
- System uptime $\geq 99.5\%$

RISK / COMPLIANCE SIGNAL

- Policy-incorrect answers per month = 0 (measured via HR audit of flagged conversations)

AGENT EXCELLENCE BLUEPRINT SUMMARY

- Purpose: Self-service HR agent for policy Q&A and routine tasks
- Users: 5,000 employees; HR generalists as indirect beneficiaries
- Agent Type: Finder (primary) + Collaborator (for actions)
- Boundaries: Answer/Ask/Act/Stop model; hard stops on privacy, legal, sensitive topics
- Knowledge: SharePoint HR policies, Benefits Guide PDF, Workday API
- Integrations: Workday (R/W), SharePoint Search (R), ServiceNow (W), Outlook (W)
- Evaluation: 6 core tests + 5 boundary tests; 5 quality signals tracked
- Operations: Weekly knowledge updates, monthly audits, quarterly policy sync
- Scale: US-only at launch $\rightarrow$ international + manager features over 9 months

Your Notes — Operations

CAPTURE YOUR THOUGHTS, QUESTIONS, AND INSIGHTS

Your Agent Excellence blueprint



Blueprint Summary

✓ Purpose & Use Case

One-sentence mission, target users, success metrics

✓ Agent Instructions

Purpose, non-goals, tone, behavior, escalation rules

✓ Boundary Model

Answer / Ask / Act / Stop classification

✓ Knowledge Plan

Sources, trust levels, refusal conditions, citations

✓ Integration Plan

Tools, read/write classification, preconditions, rollback

✓ Evaluation Set

Core test cases, boundary tests, quality signals

✓ Operating Plan

Monitor, triage, improve, communicate, scale

Use this blueprint as your reusable template for every agent you build. Return to it at each stage of design, development, and operations.

Microsoft Training resources

Power Up

During the training, you will use Copilot Studio to create solutions.

Train yourself:

- Microsoft managed curriculum, tenant and learning environment
- aka.ms/PowerUp

Train your organization:

- Tailor topics and schedule
- Incorporate business scenarios
- aka.ms/PowerUpMyTeam

Power CAT AI webinars

Engage with Microsoft experts and other customers to gain knowledge and skills needed to successfully adopt and thrive with agents.

aka.ms/PowerCAT/AIWebinars

Copilot Studio in a day

Microsoft's Power Platform in a Day workshops are immersive, hands-on training sessions designed to help participants quickly gain practical skills in using Microsoft Power Platform.

aka.ms/copilotstudioinaday

Learn for Power Platform and Copilot Studio

Structured learning paths for Microsoft Power Platform. This platform provides interactive, self-paced learning modules that cover a wide range of topics, from the fundamentals to advanced features

aka.ms/copilotstudiolearn

Copilot Studio Architecture Bootcamp

Hosted by the Microsoft Power Customer Advisory Team (Power CAT), this intensive 3-day training is designed to equip architects, technical consultants, and delivery leads with the tools and insights needed to build robust, scalable solutions.

aka.ms/CopilotStudioBootcamp

Copilot Studio guidance

Best practices, implementation and architecture guidance for planning, building, and managing agents.

aka.ms/copilotstudioguidance

Microsoft 365 Copilot Adoption

Central hub that provides guidance, resources, and best practices to help you plan, launch, and scale effective adoption of Microsoft Copilot across people, processes, and technology.

adoption.microsoft.com/copilot

The best agents aren't built by accident

They are built by teams who **make design decisions before configuration decisions.**

Today you practiced a discipline that most teams skip:
slowing down to think before building.

You learned that instructions aren't prompts. They're the operating rules that define what an agent does, how it behaves, and where it draws the line. You discovered that boundaries aren't limitations — they're what make agents trustworthy. And that knowledge design means choosing what to leave out, not just what to put in.

You wrestled with trade-offs that don't have perfect answers: usefulness versus predictability, autonomy versus control, coverage versus precision. The value isn't in finding the "right" answer — it's in making those trade-offs visible and intentional.

Every framework in this handout — the five instruction questions, the answer/ask/act/stop model, the four-stage evaluation loop — is designed to be used again. Not just for this agent, but for the next one, and the one after that.

The agents that succeed in production aren't the most sophisticated. They're the ones where someone asked the hard questions early.

Keep asking them.